

**Title:** "Real-Time Smart Sensing Dashboard with ESP32 & IoT Using Node-RED"

**Aim:** The aim is to create a real-time smart sensing system using ESP32, IoT, and Node-RED, simulating and monitoring sensor data via an interactive dashboard.

---

Resources needed: Internet Connection, ESP32, Temperature Sensor, Motion Sensor, Light Sensor, Relays, Motors, Switches, Wokwi Simulator, Power Supply, Wires & Connectors

---

### Theory:

### Pre Lab/ Prior Concepts:

Before starting the experiment, it is important to have a basic understanding of the **ESP32** microcontroller, its features, and how it connects to the internet for IoT applications. Familiarity with **IoT** concepts, including data collection, processing, and remote monitoring, is essential. Knowledge of **Node-RED** will help in building flow-based applications and creating a dashboard to visualize sensor data in real time. Understanding how to interface common **sensors** such as temperature, motion, and light sensors with microcontrollers is crucial, as well as the operation of **actuators** like relays and motors for controlling external devices. Prior experience with the **Wokwi Simulator** will be helpful for simulating circuits before physical implementation. Additionally, knowledge of **Wi-Fi** connectivity and **real-time data communication** between the microcontroller and the dashboard is key for successful project execution. Understanding **power management** for the ESP32 and other components is also important for ensuring stable operation throughout the experiment.

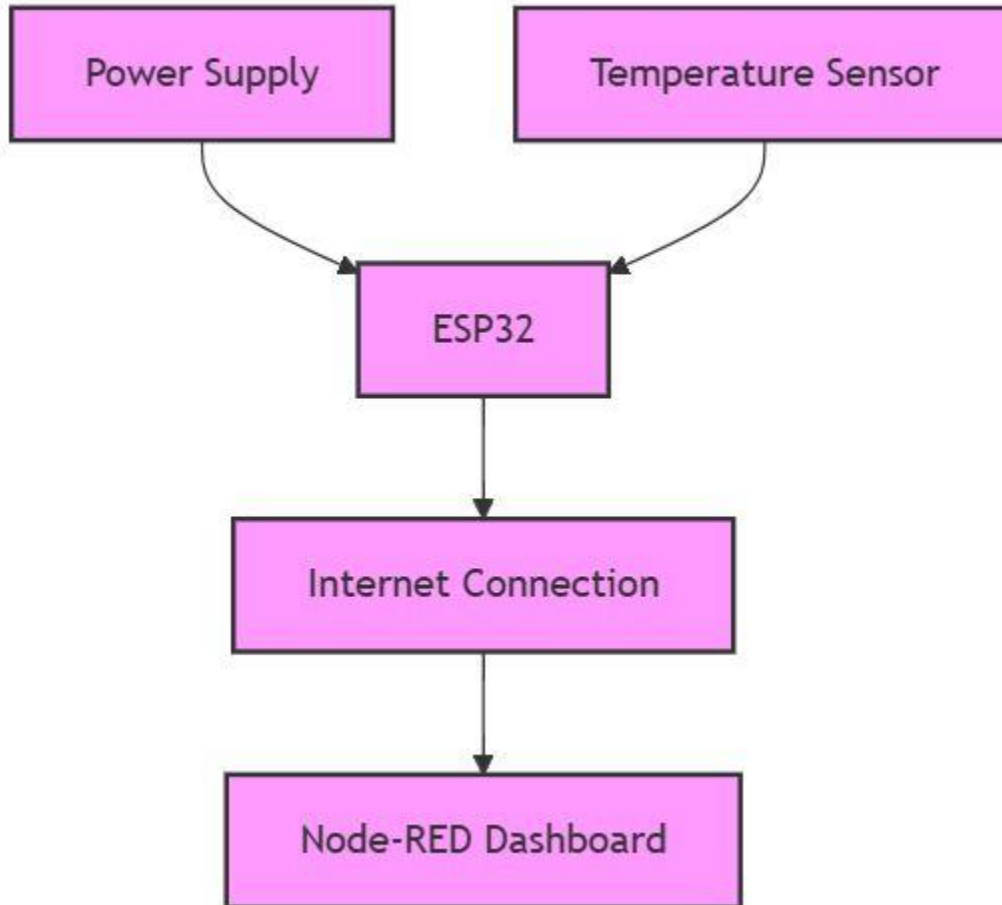
### Components of the Platform:

---

1. **ESP32 Microcontroller:** The central unit for processing and controlling the system, with built-in Wi-Fi and Bluetooth capabilities for IoT communication.
2. **Sensors:** Devices like temperature, motion, and light sensors that collect data from the environment and send it to the ESP32 for processing.
3. **Actuators:** Relays, motors, and switches that can be controlled by the ESP32 based on sensor data or commands from the dashboard.
4. **Node-RED:** A flow-based development tool for wiring together hardware devices, APIs, and online services to create a dashboard for real-time monitoring and control of the system.
5. **Wokwi Simulator:** An online platform for simulating the hardware components and their interaction, allowing testing and troubleshooting without physical components.
6. **Internet Connection:** Provides the necessary network access for communication between the ESP32 and the dashboard, enabling real-time data transmission.
7. **Power Supply:** Supplies the necessary power to the ESP32 and connected components for their operation.

8. **Wires & Connectors:** Physical components used to wire and connect the sensors, actuators, and the ESP32 together, ensuring proper communication and power distribution.
- 

### Diagram Representation



### Steps to Perform Experiments Using Wokwi

1. **Open Wokwi**
  - Visit [Wokwi](https://wokwi.com) in your web browser.
2. **Create a New Project**
  - Click on "**Start a New Project**" or choose a template for Arduino, ESP32, or Raspberry Pi.
3. **Add Components**
  - Use the "**Parts**" panel to add microcontrollers, sensors, actuators, and other electronic components.
4. **Connect the Circuit**
  - Drag and connect the components using virtual wires, ensuring proper pin connections.
5. **Write the Code**
  - Use the **built-in code editor** to write your program in C/C++ (for Arduino/ESP32) or Python (for Raspberry Pi).

## 6. Simulate the Experiment

- Click the **"Start Simulation"** button to test the circuit and observe output.

## 7. Monitor Serial Output

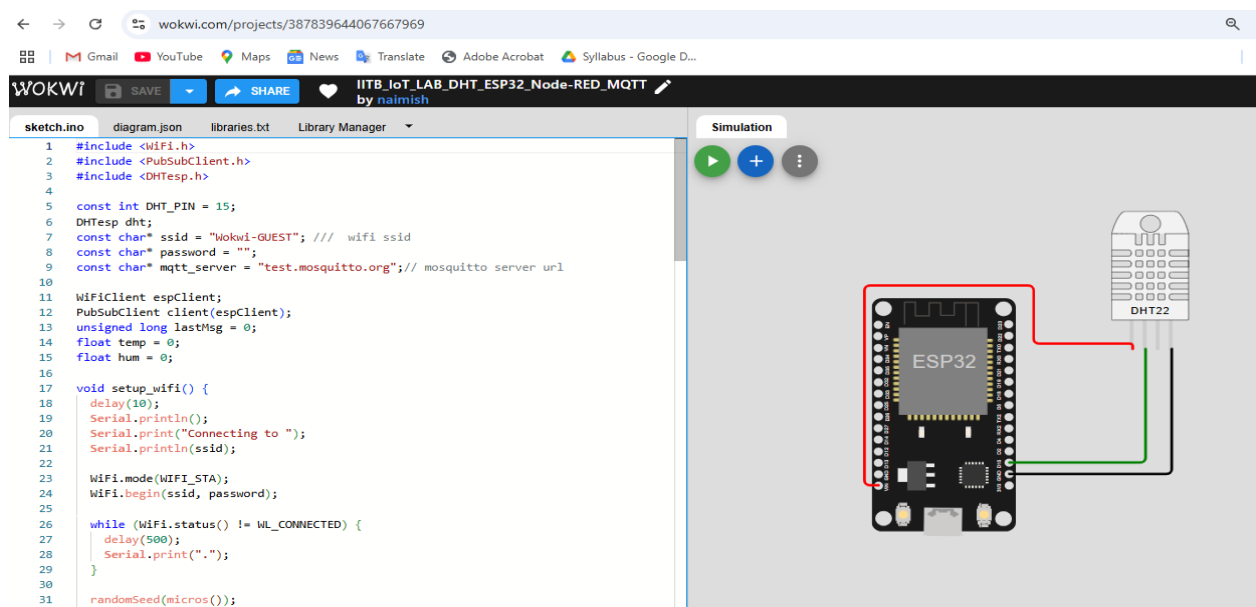
- Open the **"Serial Monitor"** to debug and view real-time data.

## 8. Modify and Optimize

- Make changes to the circuit or code based on test results and rerun the simulation.

## 9. Save and Share

- Click **"Save Project"**, then copy the project link to share or access it later.



## 1. Install Node.js (Windows)

- Visit the Node.js website: [Node.js Downloads](https://nodejs.org/en/download/)
- Download the **LTS version** (Long-Term Support).
- Follow the installation instructions. Once installed, it should add Node.js and npm (Node package manager) to your system PATH automatically.

## 2. Install Node-RED on Windows

### 1. Open Command Prompt (as Administrator if possible):

- Press Win + X and select **Command Prompt (Admin)** or **Windows PowerShell (Admin)**.

## 2. Install Node-RED Globally:

- In the Command Prompt, run the following command:

```
npm install -g --unsafe-perm node-red
```

- 3. This will install Node-RED globally on your system.

## 3. Run Node-RED

### 1. Start Node-RED:

- After installation, simply type the following command to run Node-RED:

```
node-red
```

### 2. Access the Node-RED Editor:

- Open your web browser and visit:

```
http://localhost:1880
```

- 3. This will open the Node-RED flow editor.

## 4. Installing MQTT Nodes for Node-RED

### 1. Install MQTT Nodes:

- In the Node-RED editor, click the **menu** in the top-right corner (three horizontal lines).
- Go to **Manage palette > Install** tab.
- Type node-red-node-mqtt in the search bar and click **Install** next to the result.

This installs the MQTT input and output nodes for communication with MQTT brokers.

## 5. Start Building Flows in Node-RED

Now that you have Node-RED installed, you can create flows, connect MQTT nodes, and start working with your devices.

## 6. Create a New Flow

1. Open the Node-RED editor at <http://localhost:1880>.
2. Drag and drop the following nodes:
  - **MQTT Input Node** (for subscribing to topics like `/IITB_IoT_LAB/temp` and `/IITB_IoT_LAB/hum`).

- **Debug Node** (to monitor incoming data).
- **Inject Node** (optional, for testing or triggering data).

## 7. Configure the MQTT Input Nodes

1. Double-click the **MQTT Input Node** for temperature.
  - Configure the **Broker URL** (e.g., test.mosquitto.org).
  - Set the **Topic** to /IITB\_IoT\_LAB/temp (or any topic you want to subscribe to).
2. Repeat the above for the **humidity topic**, setting the **Topic** to /IITB\_IoT\_LAB/hum.

## 8. Add Debug Nodes

1. Drag a **Debug Node** to the workspace.
2. Connect the **MQTT Input Node** for temperature to the Debug Node.
3. Connect the **MQTT Input Node** for humidity to a separate Debug Node.
4. Configure the **Debug Nodes** to show the complete message (msg.payload).

## 9. Deploy the Flow

1. Click the **Deploy** button in the top-right corner to activate your flow.

## 10. Monitor the Data in the Debug Window

1. Open the **Debug** tab in the right sidebar.
2. You should now see incoming temperature and humidity data from the ESP32 in real-time.

## 11. Optional: Add UI Dashboard for Visualization

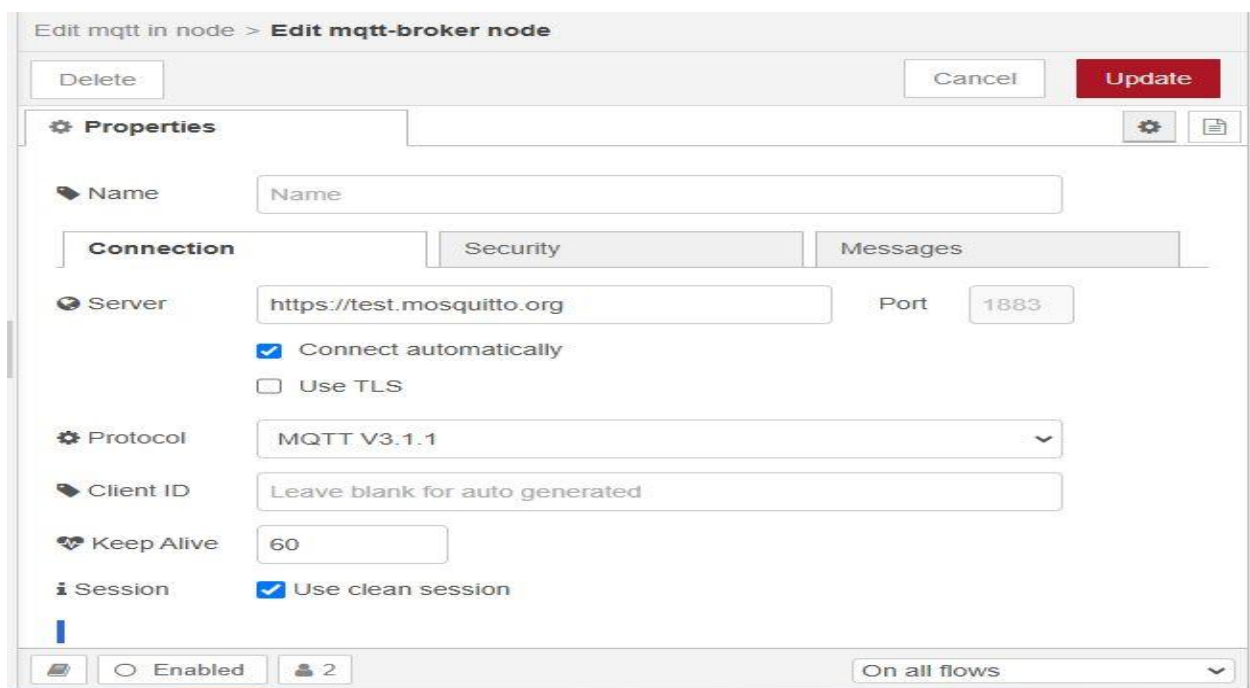
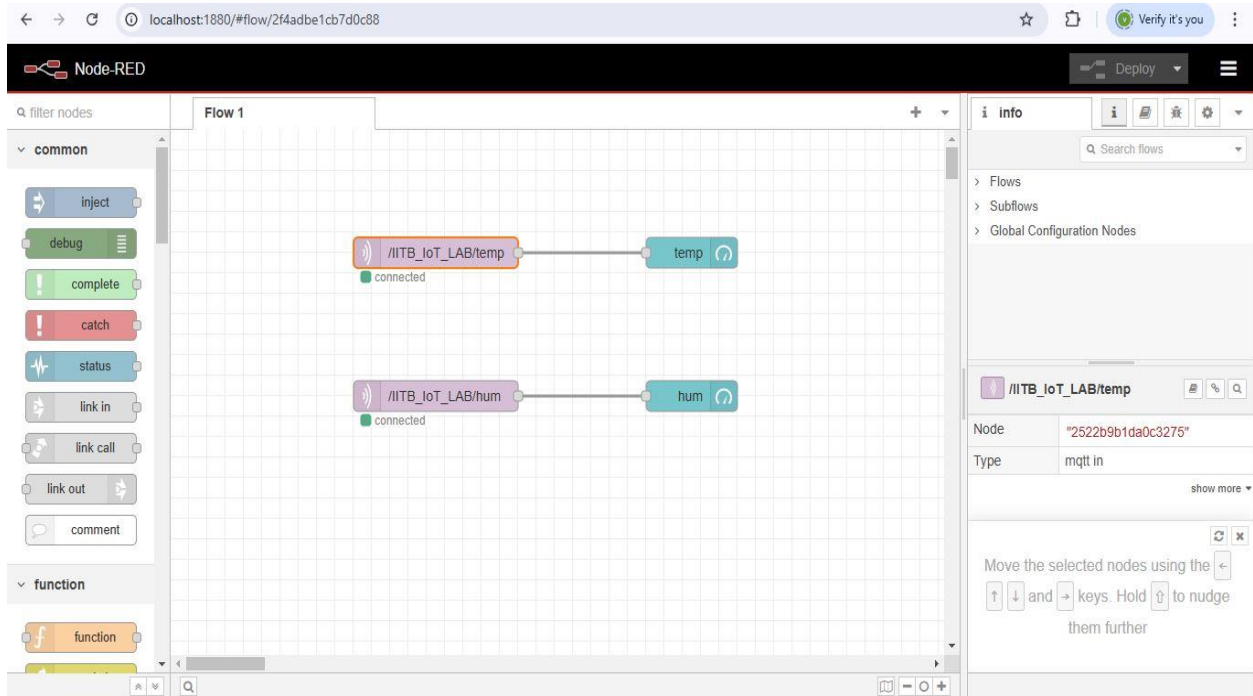
1. Drag **UI Text** or **UI Gauge** nodes from the sidebar to display temperature and humidity values.
2. Connect the **MQTT Input Nodes** to the **UI Text/Gauge Nodes**.
3. Click **Deploy** again.
4. Visit <http://localhost:1880/ui> to view the dashboard.

## 12. Optional: Store Data or Trigger Actions

1. Add **Database nodes** (e.g., **SQLite**, **InfluxDB**) to store data.
2. Use **Function Nodes** to trigger actions (e.g., turn on a fan if the temperature exceeds a threshold).

### 13. Test and Debug

1. Use the **Inject Node** to simulate sending MQTT messages to your topics.
2. Verify that messages are received correctly in the **Debug Panel**.



Delete

Cancel

Done

Properties

Server

https://test.mosquitto.org

+

Action

Subscribe to single topic

Topic

/IITB\_IoT\_LAB/temp

QoS

0

Output

auto-detect (parsed JSON object, string or buf

Name

Name

info

Search flows

Flows

Flow 1

Subflows

Global Configuration Nodes

/IITB\_IoT\_LAB/temp

Node

"2522b9b1da0c3275"

Type

mqtt in

show more

Hold down  when you 

click

 on a node to also select all of its connected nodes

Delete

Cancel

Done

Properties

Group

[Tab1] Group1

+

Size

auto

Type

Gauge

Label

temp

Value format

{{value}}

Units

units

Range

min

0

max

10

Colour gradient

Sectors

0

...

optional

...

optional

...

10

info

Search flows

Flows

Flow 1

Subflows

Global Configuration Nodes

temp

Node

"9ed84a98efb3925b"

Type

ui\_gauge

show more

Export the selected nodes, or the current tab with 

ctrl-e

© 2026, Published by Carnot Research for TSSC – ToT South Africa Training



Edit gauge node

Delete

Cancel

Done

⚙️ Properties

⚙️ 📄 🖨️

📊 Group

[Tab1] Group1

✎️ +

📏 Size

auto

☰ Type

Gauge

▼

🏷️ Label

hum

🏷️ Value format

{{value}}

🏷️ Units

units

Range

min

0

max

10

Colour gradient

Sectors

0

.....

optional

...

optional

...

10

📄

☐ Enabled

Deploy ▼

📊

Full

Deploys everything in the workspace

📊

Modified Flows

Only deploys flows that contain changed nodes

📊

Modified Nodes

Only deploys nodes that have changed

🔄

Restart Flows

Restarts the current deployed flows

Deploy ▼

☰

dashboard

📄

🔍

⚙️

▼

Layout

Site

Theme

🔗

Tabs & Links

⬆️ ⬆️ + tab + link

▼ 📊 Tab1

> 📊 Group1

Tab1

Group1

